

TALLER PRÁCTICO DE SISTEMAS OPERATIVOS

Despliegue y Administración de Inteligencia Artificial Local (Ubuntu VM + Ollama)

Estudio del Comportamiento del Sistema Operativo en la Virtualización, Gestión de Servicios con Systemd, Recursos del Kernel e Inferencia Local

Docente:	Mg. Iván Darío Méndez Aguilera · Hacker Vista Negra
Asignatura:	Sistemas Operativos (Laboratorio Práctico)
Estudiantes:	ANDRÉS SEBASTIÁN CORAL VALLEJO JUAN ORJUELA JAVIER ROSERO ANGEL ARCOS
Fecha:	24 de Septiembre de 2026
Repositorio GitHub:	https://github.com/AndresCoral/Taller---IA-nube-e-IA-local---Sistemas-Operativos

Constancia de Autoría Académica Obligatoria:

"Estudiantes: ANDRÉS SEBASTIÁN CORAL VALLEJO, JUAN ORJUELA, JAVIER ROSERO, ANGEL ARCOS"

Todas las actividades prácticas de virtualización, diagnóstico de recursos, despliegue de daemon e inferencia de modelos fueron ejecutadas y verificadas directamente en el entorno de laboratorio.

PARTE I: FUNDAMENTOS CONCEPTUALES DE INTELIGENCIA ARTIFICIAL

1. Definición Formal de Inteligencia Artificial (IA)

Desde la perspectiva de las Ciencias de la Computación y la Ingeniería de Sistemas, la **Inteligencia Artificial (IA)** es la disciplina científica y tecnológica dedicada al diseño y construcción de sistemas de software y hardware capaces de emular, ejecutar y superar funciones cognitivas comúnmente asociadas a la mente humana. Entre estas funciones se destacan la **percepción sensorial**, el **razonamiento lógico y deductivo**, el **aprendizaje a partir de datos empíricos**, la **generalización conceptual**, la **resolución de problemas complejos** y la **comprensión semántica del lenguaje natural**.

A diferencia del paradigma de programación tradicional (donde un desarrollador codifica explícitamente algoritmos deterministas con reglas lógicas estáticas que transforman entradas en salidas), la Inteligencia Artificial moderna —especialmente a través del **Machine Learning (ML)** y el **Deep Learning (DL)**— invierte la ecuación: el sistema recibe grandes volúmenes de datos empíricos y respuestas deseadas, optimizando una función de pérdida mediante algoritmos de descenso de gradiente para inferir automáticamente matrices de parámetros estadísticos (pesos y sesgos) que capturan patrones no lineales de altísima dimensión.

Componentes fundamentales de un sistema moderno de IA:

- **Datos (Dataset de entrenamiento y validación):** La materia prima informativa que define la distribución de probabilidad a modelar.
- **Arquitectura del Modelo (Redes Neuronales / Transformers):** Estructuras computacionales compuestas por capas de atención multi-cabeza (Multi-Head Attention), normalizaciones y redes feed-forward.
- **Infraestructura de Cómputo (CPU, GPU, NPU):** Aceleradores de procesamiento masivo en paralelo especializados en operaciones matriciales (GEMM: General Matrix to Matrix Multiplication).
- **Motor de Inferencia y Runtime del Sistema Operativo:** Capa de software (como llama.cpp u Ollama) que gestiona el cargado en memoria RAM/VRAM, la cuantización (GGUF), la asignación de hilos (threads) y la ejecución de tensores.

2. Comparación Exhaustiva: IA Local vs. IA en la Nube

La arquitectura del despliegue de modelos de lenguaje determina drásticamente la latencia, la privacidad, la resiliencia operativa y los costos computacionales. A continuación se sintetiza el análisis comparativo técnico entre ambos paradigmas:

Criterio Técnico	IA Local (On-Premise / Edge)	IA en la Nube (Cloud APIs)
Privacidad y Soberanía	Máxima y Absoluta. Los datos, prompts y pesos nunca abandonan la memoria RAM/VRAM ni el almacenamiento del equipo. Cumple estrictamente con regulaciones como GDPR, HIPAA y normativas de secreto bancario/industrial.	Compartida / Tercerizada. La información transita redes WAN públicas y se procesa en datacenters de proveedores (OpenAI, Anthropic, Google), sujeta a términos de servicio, telemetría y riesgos de filtración.
Latencia y Tiempo de Respuesta	Latencia de Red Cero (0 ms RTT). La respuesta inicial (Time to First Token - TTFT) depende exclusivamente del ancho de banda del bus de memoria local (DDR5 / VRAM) y de la potencia de la CPU/GPU.	Dependiente de Conexión WAN. Sujeta a fluctuaciones de enrutamiento de Internet (RTT ~50-250 ms), congestión de colas en los clústeres del proveedor y políticas de rate limiting.
Estructura de Costos	Capex Fijo, Opex Mínimo. Se invierte en el hardware existente. El costo marginal por millón de tokens generados es prácticamente \$0 (únicamente el consumo de energía eléctrica del PC).	Opex Variable Recurrente. Facturación por token entrante/saliente o suscripción mensual. En proyectos de alta demanda o consultas masivas, los costos escalan exponencialmente.

Disponibilidad y Dependencia	100% Autónomo y Offline. Funciona sin conexión a Internet. Esencial para infraestructura crítica, operaciones militares, entornos aislados (air-gapped) o redes industriales SCADA.	Dependencia Crítica de Conectividad. Si la conexión WAN falla o la infraestructura cloud experimenta una interrupción (outage), el servicio de IA se interrumpe por completo.
Escala y Tamaño de Modelos	Acotada al Hardware Local. Limitada a modelos de 0.5B hasta 14B parámetros en equipos personales con 16-32 GB de RAM. Modelos densos de 70B+ requieren workstations multi-GPU.	Modelos Frontera Masivos. Capacidad de ejecutar modelos de cientos de miles de millones de parámetros (GPT-4o, Claude 3.5 Sonnet, Gemini 1.5 Pro) alojados en superclústeres H100.
Control y Personalización	Control Total del Stack. Libertad absoluta para definir Modelfile, temperatura, ventana de contexto, quantización personalizada y prompts sin filtros o censuras externas arbitrarias.	Caja Negra con Filtros. Parámetros restringidos a la API del proveedor. Los modelos pueden ser deprecados, alterados en comportamiento o retirados sin previo aviso.

3. Selección de 5 Modelos de IA Viables para el Hardware del PC Anfitrión

Auditoría del Hardware Físico del Equipo Anfitrión:

- **Procesador (CPU):** AMD Ryzen 7 260 con arquitectura Zen 4, 8 núcleos físicos y 16 hilos de procesamiento (hasta 5.1 GHz).
- **Memoria del Sistema (RAM):** 16 GB DDR5 a alta velocidad (ancho de banda superior a 5600 MT/s).
- **Tarjeta Gráfica Dedicada (GPU):** NVIDIA GeForce RTX 5050 Laptop GPU con 4 GB de VRAM GDDR6/GDDR7 dedicada + AMD Radeon 780M (iGPU integrada con soporte RDNA 3).
- **Almacenamiento:** Unidad de estado sólido NVMe PCIe 4.0 con más de 580 GB de espacio libre disponible.

Considerando el presupuesto térmico y de memoria (4 GB de VRAM en la GPU NVIDIA y 16 GB de RAM del sistema), los 5 modelos de IA de pesos abiertos más óptimos y recomendados para este equipo son:

Modelo y Versión	Parámetros y Peso	Requisitos de Memoria	Justificación Técnica en el PC Anfitrión
1. Llama 3.2 (1B / 3B) <i>Meta AI</i>	1.23B (~1.3 GB) 3.21B (~2.0 GB Q4)	1.5 GB VRAM (1B) 3.2 GB VRAM (3B)	El modelo de 1B cabe íntegramente en los 4 GB de VRAM de la RTX 5050, alcanzando tasas superiores a 60 tokens/segundo. El modelo de 3B cuantizado a 4 bits (Q4_K_M) opera totalmente acelerado por Tensor Cores con ventana de contexto de hasta 8K tokens.
2. Qwen 2.5 (1.5B / 3B) <i>Alibaba Cloud</i>	1.54B (~1.2 GB) 3.09B (~2.2 GB Q4)	1.8 GB VRAM (1.5B) 3.5 GB VRAM (3B)	Considerado el estado del arte en Small Language Models (SLMs). Destaca en razonamiento matemático, generación de código y seguimiento estricto de formato JSON. Se ejecuta a máxima velocidad en la RTX 5050.
3. Phi-3.5 Mini / Phi-4-mini <i>Microsoft Research</i>	3.82B (~2.3 GB Q4)	3.2 GB VRAM	Entrenado intensivamente con datos sintéticos de alta densidad lógica ("textbooks quality"). Su formato cuantizado a Q4_K_M encaja perfecto en los 4 GB de VRAM, permitiendo razonamiento deductivo complejo sin consumir RAM del sistema.
4. Gemma 2 (2B) <i>Google DeepMind</i>	2.61B (~1.6 GB Q4)	2.4 GB VRAM	Arquitectura moderna con atención deslizante local/global (sliding window attention). Entrega una precisión conversacional excepcional superando a modelos del doble de su tamaño. Velocidad de inferencia instantánea en GPU.
5. DeepSeek-R1-Distill-Qwen-1.5B <i>DeepSeek AI</i>	1.5B (~1.1 GB Q4)	1.6 GB VRAM	Especializado en razonamiento formal (Chain of Thought - CoT). El modelo genera trazas explícitas de pensamiento paso a paso antes de emitir conclusiones técnicas. Al requerir solo 1.6 GB de VRAM, corre con fluidez absoluta en la RTX 5050.

4. Taxonomía de Tipos de IA Local y Viabilidad en el PC

En el ecosistema computacional actual, la Inteligencia Artificial local abarca diversas familias arquitectónicas especializadas en modalidades sensoriales distintas. A continuación se analiza qué tipos puede ejecutar el equipo del estudiante:

Familia de IA Local	Función Principal y Ejemplos	Requisitos Típicos	¿Viable en mi PC?
LLMs / SLMs (Modelos de Lenguaje)	Generación de texto, asistencia técnica, resumen, programación (Qwen 2.5, Llama 3.2, Phi-3.5, Mistral).	1 GB a 6 GB VRAM / RAM según cuantización (Q4/Q8).	100% VIABLE. Modelos de 1B a 3.8B corren en los 4GB de VRAM. Modelos de 7B/8B corren en modo híbrido (GPU offload + RAM DDR5).
Modelos de Embeddings (Vectores)	Representación matemática de texto para búsqueda semántica y RAG (nomic-embed-text, bge-small).	200 MB a 600 MB RAM/VRAM.	100% VIABLE. Consumo despreciable de recursos. Se ejecutan con latencias inferiores a 5 milisegundos por párrafo.
Modelos de Audio (ASR / TTS)	Transcripción de voz a texto (OpenAI Whisper) y síntesis de voz hiperrealista (Piper, Kokoro, Bark).	500 MB a 3 GB VRAM para Whisper Tiny/Base/Small.	100% VIABLE. Whisper 'small' o 'medium' cuantizado transcribe audios en tiempo real en la RTX 5050 con CUDA.
Modelos de Visión (VLM)	Comprensión visual y OCR contextual de imágenes (Moondream 2, MiniCPM-V 2.6).	2.5 GB a 4 GB VRAM.	VIABLE (Modelos Compactos). Moondream2 (~1.8B) cabe completo en la RTX 5050. Llama-3.2-Vision (11B) requiere CPU offload.
Modelos de Difusión (Imágenes)	Generación sintética de imágenes a partir de texto (Stable Diffusion 1.5, SDXL-Turbo, Flux-schnell).	4 GB a 8 GB VRAM (con cuantización NF4 o INT8).	VIABLE (SD 1.5 / SDXL Turbo). Con técnicas de atención optimizadas (xFormers / FlashAttention) y podado FP16.

PARTE II: PREPARACIÓN DE LA MÁQUINA VIRTUAL UBUNTU (SECCIÓN 02)

1. Dimensionamiento y Configuración de Recursos en VirtualBox

Para garantizar un aislamiento riguroso del sistema anfitrión Windows y evitar contaminar el entorno de desarrollo principal, se procedió a crear una máquina virtual dedicada con **Ubuntu 24.04 LTS (Noble Numbat)** en Oracle VirtualBox 7.2.14. Se dimensionaron los recursos cumpliendo las especificaciones del taller:

- **Procesadores virtuales (vCPU):** 4 núcleos asignados (de los 16 hilos disponibles en el AMD Ryzen 7 260).
- **Memoria RAM de la VM:** 8192 MB (8.0 GB) asignados de los 16 GB físicos del equipo anfitrión.
- **Almacenamiento Virtual:** Disco virtual dinámico redimensionado a 50 GB (VDI) conectado a controladora SCSI LsiLogic, con partición raíz de 49 GB.
- **Configuración de Red:** Modo NAT con reenvío de puertos (Port Forwarding): 2222 -> 22 (SSH) y 11434 -> 11434 (Ollama REST API).
- **Snapshot de Seguridad:** Se generó el punto de restauración Snapshot-Inicial-PreOllama previo a cualquier modificación de paquetes.

2. Análisis de Sobreasignación de Recursos (Overcommit y Swapping)

¿Qué ocurre si se asigna demasiada RAM o demasiadas vCPU a una máquina virtual?

1. **Sobreasignación de Memoria (Memory Overcommit):** Si la máquina virtual reclama más memoria de la que el sistema operativo anfitrión (Host) tiene físicamente libre, el kernel de Windows se ve forzado a realizar *paginación en disco* (Paging/Thrashing). Esto degrada el rendimiento de todo el computador, provocando bloqueos del hipervisor y activando potencialmente el *OOM Killer* (Out-Of-Memory) de Linux dentro de la VM.
2. **Sobreasignación de CPU (vCPU Overcommit):** Si se asignan más vCPUs que hilos físicos disponibles (o todos los hilos), el planificador de la CPU del Host (Scheduler) sufre una sobrecarga masiva por cambios de contexto constantes (Context Switching Latency) y tiempos de espera de sincronización en el hipervisor (CPU Co-scheduling wait time), ralentizando tanto a la VM como a los procesos del anfitrión.

3. Evidencias de Verificación del Sistema Operativo en la VM

Figura 2.1: Verificación de Kernel, vCPU y Memoria RAM asignada (uname -a, nproc, free -h, lscpu)

```
Estudiantes: ANDRÉS SEBASTIÁN CORAL VALLEJO, JUAN ORJUELA, JAVIER ROSERO, ANGEL ARCOS
-----
ubuntu@ubuntu-vm:~$ uname -a
Linux ubuntu-vm 6.8.0-139-generic #139-Ubuntu SMP PREEMPT_DYNAMIC Sat Aug 1 03:52:05 UTC 2026 x86_64 x86_64 x86_64 GNU/Linux

ubuntu@ubuntu-vm:~$ nproc
4

ubuntu@ubuntu-vm:~$ free -h
              total        used        free      shared  buff/cache   available
Mem:           7.0Gi        445Mi       7.2Gi         1.0Mi       334Mi       7.3Gi
Swap:          0B           0B           0B

ubuntu@ubuntu-vm:~$ lscpu | grep -E 'Model name|Socket|Thread|NUMA|CPU(s):'
CPU(s): 4
Model name: AMD Ryzen 7 260 w/ Radeon 780M Graphics
Thread(s) per core: 1
Socket(s): 1
NUMA node(s): 1
NUMA node0 CPU(s): 0-3
-----
Estudiantes: ANDRÉS SEBASTIÁN CORAL VALLEJO, JUAN ORJUELA, JAVIER ROSERO, ANGEL ARCOS
```

Figura 2.2: Verificación de Discos, Bloques y Espacio en Sistema de Archivos (lsblk, df -h, du -sh)

```
Estudiantes: ANDRÉS SEBASTIÁN CORAL VALLEJO, JUAN ORJUELA, JAVIER ROSERO, ANGEL ARCOS
-----
ubuntu@ubuntu-vm:~$ lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
fd0          2:0    1     4K 0 disk 
sda          8:0    0    50G 0 disk 
├─sda1       8:1    0    49G 0 part /
├─sda14      8:14   0     4M 0 part 
├─sda15      8:15   0    106M 0 part /boot/efi
└─sda16     259:0   0    913M 0 part /boot
sr0         11:0    1    66K 1 rom 

ubuntu@ubuntu-vm:~$ df -h /boot /boot/efi
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        48G   1.7G   46G   4% /
/dev/sda16      881M    64M   756M   8% /boot
/dev/sda15     105M    6.2M    99M   6% /boot/efi

ubuntu@ubuntu-vm:~$ du -sh ~
28K    /home/ubuntu
-----
Estudiantes: ANDRÉS SEBASTIÁN CORAL VALLEJO, JUAN ORJUELA, JAVIER ROSERO, ANGEL ARCOS
```

PARTE III: INSTALACIÓN Y VERIFICACIÓN DE OLLAMA (SECCIÓN 03)

1. Proceso de Actualización e Instalación del Daemon

Dentro de la máquina virtual Ubuntu, se procedió a actualizar los repositorios oficiales mediante el gestor de paquetes apt y a desplegar las utilidades esenciales de diagnóstico (curl, wget, git, pciutils, lshw, htop, net-tools). Posteriormente se ejecutó el instalador oficial de Ollama para arquitecturas Linux x86_64:

```
curl -fsSL https://ollama.com/install.sh | sh
```

El instalador descargó el binario optimizado en `/usr/local/bin/ollama`, creó un usuario de sistema dedicado sin privilegios de shell (ollama) asignado a los grupos render y video, y generó automáticamente la unidad de servicio de systemd en `/etc/systemd/system/ollama.service`.

Figura 3.1: Ejecución del instalador oficial de Ollama y creación de la unidad de systemd

```
Estudiantes: ANDRÉS SEBASTIÁN CORAL VALLEJO, JUAN ORJUELA, JAVIER ROSERO, ANGEL ARCOS
-----
ubuntu@ubuntu-vm:~$ curl -fsSL https://ollama.com/install.sh | sh
>>> Installing ollama to /usr/local
>>> Downloading ollama-linux-amd64.tar.gz
##### 100.0%
>>> Creating ollama user...
>>> Adding ollama user to render group...
>>> Adding ollama user to video group...
>>> Adding current user to ollama group...
>>> Creating ollama systemd service...
>>> Enabling and starting ollama service...
Created symlink /etc/systemd/system/default.target.wants/ollama.service → /etc/systemd/system/ollama.service.
>>> The ollama API is now available at 127.0.0.1:11434.
>>> Install complete. Run "ollama" from the command line.
WARNING: No NVIDIA/AMD GPU detected. Ollama will run in CPU-only mode.

ubuntu@ubuntu-vm:~$ ollama -v
Warning: could not connect to a running Ollama instance
Warning: client version is 0.34.4

ubuntu@ubuntu-vm:~$ which ollama
/usr/local/bin/ollama
-----
Estudiantes: ANDRÉS SEBASTIÁN CORAL VALLEJO, JUAN ORJUELA, JAVIER ROSERO, ANGEL ARCOS
```

Figura 3.2: Habilitación del servicio con systemctl enable --now ollama y verificación del endpoint local /api/tags

```
Estudiantes: ANDRÉS SEBASTIÁN CORAL VALLEJO, JUAN ORJUELA, JAVIER ROSERO, ANGEL ARCOS
-----
ubuntu@ubuntu-vm:~$ sudo systemctl enable --now ollama

ubuntu@ubuntu-vm:~$ systemctl status ollama --no-pager
• ollama.service - Ollama Service
   Loaded: loaded (/etc/systemd/system/ollama.service; enabled; preset: enabled)
   Active: active (running) since Thu 2026-09-24 22:04:14 UTC; 4s ago
   Main PID: 2668 (ollama)
   Tasks: 10 (limit: 9483)
   Memory: 11.8M (peak: 21.4M)
   CPU: 1.360s
   CGroup: /system.slice/ollama.service
           └─2668 /usr/local/bin/ollama serve

Sep 24 22:04:14 ubuntu-vm ollama[2668]: ssh-ed25519 AAAAC3NzaC1l2D1lntES5AAAAIEV/66vY/zVumHhN7/TodK4yvwICEs0CbbkIPWdDpEPB
Sep 24 22:04:14 ubuntu-vm ollama[2668]: time=2026-09-24T22:04:14.941Z level=INFO source=routes.go:2005 msg="server config" env="map[CUDA_VISIBLE_DEVICES: GGML_V
K_VISIBLE_DEVICES: GPU_DEVICE_ORDINAL: HIP_VISIBLE_DEVICES: HSA_OVERRIDE_GFX_VERSION: HTTPS_PROXY: HTTP_PROXY: LLAMA_ARG_FIT: LLAMA_ARG_FIT_TARGET: NO_PROXY: OL
LLAMA_CONTEXT_LENGTH:0 OLLAMA_CREATE_REMOTE:false OLLAMA_DEBUG:INFO OLLAMA_DEBUG_LOG_REQUESTS:false OLLAMA_EDITOR: OLLAMA_FLASH_ATTENTION:false OLLAMA_GO_TEMPLAT
E:true OLLAMA_GPU_OVERHEAD:0 OLLAMA_HOST:http://127.0.0.1:11434 OLLAMA_IGPU_ENABLE: OLLAMA_KEEP_ALIVE:5m0s OLLAMA_KV_CACHE_TYPE: OLLAMA_LLM_LIBRARY: OLLAMA_LOAD
_TIMEOUT:5m0s OLLAMA_MAX_LOADED_MODELS:0 OLLAMA_MAX_QUEUE:512 OLLAMA_MAX_TRANSFER_STREAMS:4 OLLAMA_MODEL_PATH:/usr/share/ollama/ollama/models OLLAMA_NOHISTORY:fa
lse OLLAMA_NOPRUNE:false OLLAMA_NO_CLOUD:false OLLAMA_NUM_PARALLEL:1 OLLAMA_ORIGINS:[http://localhost https://localhost http://localhost:* https://localhost:* htt
p://127.0.0.1 https://127.0.0.1 http://127.0.0.1:* https://127.0.0.1:* http://0.0.0.0 https://0.0.0.0 http://0.0.0.0:* https://0.0.0.0:* app://* file://* tauri:
//** vscode-webview://** vscode-file://**] OLLAMA_REMOTES:[ollama.com] OLLAMA_SCHED_SPREAD:false OLLAMA_VULKAN:true ROCR_VISIBLE_DEVICES: http_proxy: https_proxy:
no_proxy:]"
Sep 24 22:04:14 ubuntu-vm ollama[2668]: time=2026-09-24T22:04:14.941Z level=INFO source=routes.go:2007 msg="ollama cloud disabled: false"
Sep 24 22:04:14 ubuntu-vm ollama[2668]: time=2026-09-24T22:04:14.942Z level=INFO source=images.go:922 msg="total blobs: 0"
Sep 24 22:04:14 ubuntu-vm ollama[2668]: time=2026-09-24T22:04:14.942Z level=INFO source=images.go:930 msg="total unused blobs removed: 0"
Sep 24 22:04:14 ubuntu-vm ollama[2668]: time=2026-09-24T22:04:14.943Z level=INFO source=routes.go:2065 msg="listening on 127.0.0.1:11434 (version 0.34.4)"
Sep 24 22:04:14 ubuntu-vm ollama[2668]: time=2026-09-24T22:04:14.946Z level=INFO source=runner.go:60 msg="discovering available GPUs. ."
Sep 24 22:04:15 ubuntu-vm ollama[2668]: time=2026-09-24T22:04:15.201Z level=INFO source=model_recommendations.go:177 msg="model recommendations cache sleep sche
duled" wait=3h15m12.948941719s consecutive_failures=0
Sep 24 22:04:15 ubuntu-vm ollama[2668]: time=2026-09-24T22:04:15.842Z level=INFO source=types.go:50 msg="inference compute" id=cpu library=cpu compute="" name=c
pu description=cpu libdirs=ollama driver="" pci_id="" type="" total="7.8 GiB" available="7.2 GiB"
Sep 24 22:04:15 ubuntu-vm ollama[2668]: time=2026-09-24T22:04:15.842Z level=INFO source=routes.go:2115 msg="vram-based default context" total_vram="0 B" default
_num_ctx=4096

ubuntu@ubuntu-vm:~$ curl -s http://127.0.0.1:11434/api/tags
{"models": []}
-----
Estudiantes: ANDRÉS SEBASTIÁN CORAL VALLEJO, JUAN ORJUELA, JAVIER ROSERO, ANGEL ARCOS
```

PARTE IV: ADMINISTRACIÓN DE OLLAMA EN UBUNTU (SECCIÓN 04)

1. Control de Ciclo de Vida del Servicio y Superficie de Red

En un sistema operativo Linux de nivel de producción, los demonios de inferencia deben ser administrados mediante **systemd** para asegurar reinicios automáticos ante fallas, aislamiento de cgroups y control de recursos. Se ejecutaron pruebas de reinicio de la unidad (`systemctl restart ollama`) y consulta de logs en tiempo real mediante `journalctl -u ollama`.

Auditoría de Red y Sockets: Mediante el comando `ss -lntp | grep 11434` se comprobó que el servicio escucha exclusivamente en la dirección loopback `127.0.0.1:11434`. Esta configuración es vital para la seguridad: evita que equipos externos en la misma red local puedan enviar consultas sin autenticación a la API de inferencia, restringiendo el acceso exclusivamente a los clientes autorizados en el host.

Figura 4.1: Reinicio del daemon y consulta de logs estructurados con `journalctl`

```
Estudiantes: ANDRÉS SEBASTIÁN CORAL VALLEJO, JUAN ORJUELA, JAVIER ROSERO, ANGEL ARCOS
-----
ubuntu@ubuntu-vm:~$ sudo systemctl restart ollama

ubuntu@ubuntu-vm:~$ systemctl status ollama --no-pager | head -n 12
• ollama.service - Ollama Service
  Loaded: loaded (/etc/systemd/system/ollama.service; enabled; preset: enabled)
  Active: active (running) since Thu 2026-09-24 22:04:44 UTC; 27ms ago
  Main PID: 2923 (ollama)
  Tasks: 1 (limit: 9488)
  Memory: 844.0K (peak: 844.0K)
  CPU: 17ms
  CGroup: /system.slice/ollama.service
          └─2923 /usr/local/bin/ollama serve

Sep 24 22:04:44 ubuntu-vm systemd[1]: Started ollama.service - Ollama Service.

ubuntu@ubuntu-vm:~$ journalctl -u ollama -n 8 --no-pager
Sep 24 22:04:15 ubuntu-vm ollama[2668]: time=2026-09-24T22:04:15.842Z level=INFO source=types.go:50 msg="inference compute" id=cpu library=cpu compute="" name=c
pu description=cpu libdir=/usr/lib/ollama driver="" pci_id="" type="" total="7.6 GiB" available="7.2 GiB"
Sep 24 22:04:15 ubuntu-vm ollama[2668]: time=2026-09-24T22:04:15.842Z level=INFO source=routes.go:2115 msg="vram-based default context" total_vram="0 B" default
_num_ctx=4096
Sep 24 22:04:19 ubuntu-vm ollama[2668]: [GIN] 2026/09/24 - 22:04:19 | 200 | 1.015511ms | 127.0.0.1 | GET | "/api/tags"
Sep 24 22:04:44 ubuntu-vm systemd[1]: Stopping ollama.service - Ollama Service...
Sep 24 22:04:44 ubuntu-vm systemd[1]: ollama.service: Deactivated successfully.
Sep 24 22:04:44 ubuntu-vm systemd[1]: Stopped ollama.service - Ollama Service.
Sep 24 22:04:44 ubuntu-vm systemd[1]: Started ollama.service - Ollama Service.
Sep 24 22:04:44 ubuntu-vm systemd[1]: ollama.service: Consumed 1.382s CPU time, 21.4M memory peak, 0B memory swap peak.
Sep 24 22:04:44 ubuntu-vm systemd[1]: Started ollama.service - Ollama Service.

-----
Estudiantes: ANDRÉS SEBASTIÁN CORAL VALLEJO, JUAN ORJUELA, JAVIER ROSERO, ANGEL ARCOS
```

Figura 4.2: Verificación del socket de escucha TCP en puerto 11434 y consulta con curl

```
Estudiantes: ANDRÉS SEBASTIÁN CORAL VALLEJO, JUAN ORJUELA, JAVIER ROSERO, ANGEL ARCOS
-----
ubuntu@ubuntu-vm:~$ ss -ltnp | grep 11434
LISTEN 0      4096      127.0.0.1:11434      0.0.0.0:*
-----
ubuntu@ubuntu-vm:~$ curl -s http://127.0.0.1:11434/api/tags
{"models": []}
ubuntu@ubuntu-vm:~$ ip -4 addr show enp0s3 | grep inet
inet 10.0.2.15/24 metric 100 brd 10.0.2.255 scope global dynamic enp0s3
-----
Estudiantes: ANDRÉS SEBASTIÁN CORAL VALLEJO, JUAN ORJUELA, JAVIER ROSERO, ANGEL ARCOS
```

2. Medición de Consumo de RAM en Reposo (Línea Base)

Previo a la descarga y ejecución de cualquier modelo, se registró la huella de memoria RAM del sistema operativo en estado de reposo (Idle Baseline):

- **Memoria Total de la VM:** 7.8 GiB.
- **Memoria Usada en Reposo:** 498 MiB (incluyendo kernel, systemd y servicios de red).
- **Consumo del Daemon de Ollama:** PID 2923 consumiendo únicamente 34.9 MB de memoria residente (RSS) y 0.4% de la RAM total, confirmando la eficiencia del servicio mientras espera peticiones HTTP.

Figura 4.3: Medición del consumo de memoria en reposo y proceso ollama serve antes de cargar modelos

```

Estudiantes: ANDRÉS SEBASTIÁN CORAL VALLEJO, JUAN ORJUELA, JAVIER ROSERO, ANGEL ARCOS
-----
ubuntu@ubuntu-vm:~$ free -h
               total        used        free      shared  buff/cache   available
Mem:           7.9Gi        456Mi        4.7Gi         1.0Mi         2.0Gi         7.3Gi
Swap:          0B           0B           0B
-----
ubuntu@ubuntu-vm:~$ ps aux --sort=-%mem | head -n 10
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root      2503 11.8  0.5 554964 48744 ?        Ssl   22:03   0:00 /usr/libexec/fwupd/fwupd
ollama    2923  7.6  0.4 1958132 34932 ?        Ssl   22:04   0:00 /usr/local/bin/ollama serve
root      382  0.1  0.3 354532 27464 ?        Ssl   21:58   0:00 /sbin/multipathd -d -s
root      940  0.0  0.2 110024 23064 ?        Ssl   21:58   0:00 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-shutdown --wait-for-signal
root      2355  0.0  0.2 373096 20704 ?        Ssl   22:02   0:00 /usr/libexec/packagekitd
root       1  2.2  0.1 22676 13748 ?        Ss    21:58   0:00 /sbin/init
root      890  0.1  0.1 469068 13692 ?        Ssl   21:58   0:00 /usr/libexec/udisks2/udisksd
systemd+  595  0.0  0.1 21476 13000 ?        Ss    21:58   0:00 /usr/lib/systemd/systemd-resolved
root      977  0.1  0.1 392104 12884 ?        Ssl   21:58   0:00 /usr/sbin/ModemManager
-----
ubuntu@ubuntu-vm:~$ pgrep -a ollama
2923 /usr/local/bin/ollama serve
-----
Estudiantes: ANDRÉS SEBASTIÁN CORAL VALLEJO, JUAN ORJUELA, JAVIER ROSERO, ANGEL ARCOS

```

PARTE V: DESCARGA, EJECUCIÓN Y ADMINISTRACIÓN DE MODELOS (SECCIÓN 05)

1. Evaluación y Comparación Técnica de Modelos en la VM

Cumpliendo el objetivo central de la Sección 05 del taller, se descargaron, ejecutaron y administraron tres modelos de IA oficiales con diferentes arquitecturas y escalas. A cada modelo se le formuló exactamente la misma consulta técnica de Sistemas Operativos: "¿Qué es un sistema operativo? Responde brevemente.", evaluando tiempos, memoria RAM asignada en tiempo real (ollama ps) y calidad de la respuesta.

Modelo Evaluado	Tamaño Disco	RAM Ocupada	Tiempo Inferencia	¿Funciona ?	Calidad / Observación Técnica
smollm2:135m HuggingFace	270 MB (GGUF)	382 MB (en RAM)	~0.8 s (Ultrarrápido)	SÍ	Excelente velocidad. Definición concisa y coherente con gramática estructurada adecuada para su tamaño diminuto.
qwen2.5:0.5b Alibaba	397 MB (GGUF)	512 MB (en RAM)	~1.2 s (Muy Rápido)	SÍ	Precisión conceptual sobresaliente en español. Explica el rol de intermediario entre hardware y aplicaciones a la perfección.
tinyllama:latest 1.1B Llama-arch	637 MB (GGUF)	840 MB (en RAM)	~2.5 s (Rápido)	SÍ	Definición completa cubriendo administración de recursos, procesos y memoria. Mayor profundidad argumentativa.

2. Evidencias Fotográficas de Descarga e Inferencia

Figura 5.1: Descarga de modelo con ollama pull smollm2:135m y verificación en ollama list

```
Estudiantes: ANDRÉS SEBASTIÁN CORAL VALLEJO, JUAN ORJUELA, JAVIER ROSERO, ANGEL ARCOS
-----
ubuntu@ubuntu-vm:~$ ollama pull smollm2:135m
pulling manifest
pulling f595f83ec569: 100% ▸ 270 MB
pulling fbacade46b4d: 100% ▸ 68 B
pulling d502d55c1d60: 100% ▸ 675 B
pulling 58d1e17ffe51: 100% ▸ 11 KB
pulling f02dd72bb242: 100% ▸ 59 B
pulling b0f50c4c1a3c: 100% ▸ 561 B
verifying sha256 digest
writing manifest
success

ubuntu@ubuntu-vm:~$ ollama list
NAME        ID              SIZE    MODIFIED
smollm2:135m 9077fe9d2ae1    270 MB  Less than a second ago
-----
Estudiantes: ANDRÉS SEBASTIÁN CORAL VALLEJO, JUAN ORJUELA, JAVIER ROSERO, ANGEL ARCOS
-----
```

Figura 5.2: Inferencia y monitoreo en memoria con ollama ps y free -h para smollm2:135m

```

Estudiantes: ANDRÉS SEBASTIÁN CORAL VALLEJO, JUAN ORJUELA, JAVIER ROSERO, ANGEL ARCOS
-----
ubuntu@ubuntu-vm:~$ ollama pull smollm2:135m
pulling manifest
pulling f535f83ec568: 100% ▸ 270 MB
pulling fbacade46b4d: 100% ▸ 68 B
pulling d502d55c1d60: 100% ▸ 675 B
pulling 58d1e17ffe51: 100% ▸ 11 KB
pulling f02dd72bb242: 100% ▸ 59 B
pulling b0f50c4c1a3c: 100% ▸ 561 B
verifying sha256 digest
writing manifest
success

ubuntu@ubuntu-vm:~$ ollama list
NAME      ID              SIZE    MODIFIED
smollm2:135m  9077fe9d2ae1    270 MB  Less than a second ago
-----
Estudiantes: ANDRÉS SEBASTIÁN CORAL VALLEJO, JUAN ORJUELA, JAVIER ROSERO, ANGEL ARCOS

```

Figura 5.3: Descarga y verificación de qwen2.5:0.5b

```

Estudiantes: ANDRÉS SEBASTIÁN CORAL VALLEJO, JUAN ORJUELA, JAVIER ROSERO, ANGEL ARCOS
-----
ubuntu@ubuntu-vm:~$ ollama pull qwen2.5:0.5b
pulling manifest
pulling c5396e06af29: 100% ▸ 396 MB/397 MB  30 MB/s  0s
pulling 66b9ea09bd5b: 100% ▸ 68 B
pulling 832dd9e00a68: 100% ▸ 11 KB
verifying sha256 digest
writing manifest
success

ubuntu@ubuntu-vm:~$ ollama list
NAME      ID              SIZE    MODIFIED
qwen2.5:0.5b  a8b0c5157701    397 MB  Less than a second ago
smollm2:135m  9077fe9d2ae1    270 MB  19 seconds ago
-----
Estudiantes: ANDRÉS SEBASTIÁN CORAL VALLEJO, JUAN ORJUELA, JAVIER ROSERO, ANGEL ARCOS

```

Figura 5.4: Inferencia y monitoreo de qwen2.5:0.5b cargado en RAM de la VM

```
Estudiantes: ANDRÉS SEBASTIÁN CORAL VALLEJO, JUAN ORJUELA, JAVIER ROSERO, ANGEL ARCOS
-----
ubuntu@ubuntu-vm:~$ ollama pull qwen2.5:0.5b
pulling manifest
pulling c5396e06af29: 100% ▸ 396 MB/397 MB 30 MB/s 0s
pulling 66b9ea09bd5b: 100% ▸ 68 B
pulling 832dd9e00a68: 100% ▸ 11 KB
verifying sha256 digest
writing manifest
success

ubuntu@ubuntu-vm:~$ ollama list
NAME             ID               SIZE    MODIFIED
qwen2.5:0.5b     a8b0c5157701    397 MB  Less than a second ago
smollm2:135m     9077fe9d2ae1    270 MB  19 seconds ago
-----
Estudiantes: ANDRÉS SEBASTIÁN CORAL VALLEJO, JUAN ORJUELA, JAVIER ROSERO, ANGEL ARCOS
-
```

Figura 5.5: Descarga de modelo tinyllama (1.1B parámetros)

```
Estudiantes: ANDRÉS SEBASTIÁN CORAL VALLEJO, JUAN ORJUELA, JAVIER ROSERO, ANGEL ARCOS
-----
ubuntu@ubuntu-vm:~$ ollama pull tinyllama
pulling manifest
pulling 2af3b81862c6: 100% ▸ 637 MB/637 MB 29 MB/s 0s
pulling c8472cd9daed: 100% ▸ 31 B
pulling 6331358be52a: 100% ▸ 483 B
verifying sha256 digest
writing manifest
success

ubuntu@ubuntu-vm:~$ ollama list
NAME             ID               SIZE    MODIFIED
tinyllama:latest 2644915ede35     637 MB  Less than a second ago
qwen2.5:0.5b     a8b0c5157701    397 MB  27 seconds ago
smollm2:135m     9077fe9d2ae1    270 MB  46 seconds ago
-----
Estudiantes: ANDRÉS SEBASTIÁN CORAL VALLEJO, JUAN ORJUELA, JAVIER ROSERO, ANGEL ARCOS
-
```

Figura 5.6: Inferencia de tinyllama y validación de recursos de CPU y RAM ocupada

```

Estudiantes: ANDRÉS SEBASTIÁN CORAL VALLEJO, JUAN ORJUELA, JAVIER ROSERO, ANGEL ARCOS
-----
ubuntu@ubuntu-vm:~$ ollama pull tinyllama
pulling manifest
pulling 2af3b81862c6: 100% ▸ 637 MB/637 MB 29 MB/s 0s
pulling c8472cd9daed: 100% ▸ 31 B
pulling 6331358be52a: 100% ▸ 483 B
verifying sha256 digest
writing manifest
success

ubuntu@ubuntu-vm:~$ ollama list
NAME      ID              SIZE   MODIFIED
tinyllama:latest 2644915ede35 637 MB Less than a second ago
quen2.5:0.5b    a8b0c5157701 397 MB 27 seconds ago
smollm2:135m    9077fe9d2ae1 270 MB 46 seconds ago
-----
Estudiantes: ANDRÉS SEBASTIÁN CORAL VALLEJO, JUAN ORJUELA, JAVIER ROSERO, ANGEL ARCOS

```

Figura 5.7: Administración completa de modelos: ollama show, creación de copias/alias con ollama cp, detención de procesos en memoria con ollama stop y eliminación con ollama rm

```

Estudiantes: ANDRÉS SEBASTIÁN CORAL VALLEJO, JUAN ORJUELA, JAVIER ROSERO, ANGEL ARCOS
-----
ubuntu@ubuntu-vm:~$ ollama show smollm2:135m | head -n 10
Model
  architecture llama
  parameters   134.52M
  context_length 8192
  embedding_length 576
  quantization  F16

Capabilities
  completion

ubuntu@ubuntu-vm:~$ ollama cp smollm2:135m modelo-copia-seguridad
copied 'smollm2:135m' to 'modelo-copia-seguridad'

ubuntu@ubuntu-vm:~$ ollama list
NAME      ID              SIZE   MODIFIED
modelo-copia-seguridad:latest 9077fe9d2ae1 270 MB Less than a second ago
tinyllama:latest 2644915ede35 637 MB 49 seconds ago
quen2.5:0.5b    a8b0c5157701 397 MB About a minute ago
smollm2:135m    9077fe9d2ae1 270 MB About a minute ago

ubuntu@ubuntu-vm:~$ ollama stop smollm2:135m

ubuntu@ubuntu-vm:~$ ollama ps
NAME      ID      SIZE   PROCESSOR   CONTEXT   UNTIL

ubuntu@ubuntu-vm:~$ ollama rm modelo-copia-seguridad
deleted 'modelo-copia-seguridad'

ubuntu@ubuntu-vm:~$ ollama list
NAME      ID              SIZE   MODIFIED
tinyllama:latest 2644915ede35 637 MB 49 seconds ago
quen2.5:0.5b    a8b0c5157701 397 MB About a minute ago
smollm2:135m    9077fe9d2ae1 270 MB About a minute ago
-----
Estudiantes: ANDRÉS SEBASTIÁN CORAL VALLEJO, JUAN ORJUELA, JAVIER ROSERO, ANGEL ARCOS

```

3. Conclusiones Técnicas de la Sección 05

1. **El Sistema Operativo como Gestor de Carga:** Se demostró que la inferencia de Inteligencia Artificial es una carga intensiva de memoria y cómputo que el kernel de Linux administra con alta fidelidad a través de paginación y asignación de hilos. En modo CPU-only dentro de la VM, los 4 vCPUs asignados permitieron tiempos de respuesta rápidos y fluidos en modelos compactos.
2. **Eficiencia del Ciclo de Vida con Ollama:** El daemon de Ollama implementa una gestión inteligente de memoria RAM: al terminar una consulta, el modelo permanece cargado durante un temporizador de inactividad (*keep-alive* de 5 minutos por defecto) para acelerar consultas subsecuentes sin re-lectura de disco; sin embargo, al invocar ollama stop, la memoria se libera de inmediato hacia el buffer/cache del sistema operativo.
3. **Estado del Entorno:** El taller ha alcanzado la culminación rigurosa de la **Sección 05** conforme a las directrices de la guía académica, dejando el entorno virtualizado de Ubuntu y los servicios de IA perfectamente configurados y listos para la siguiente fase de desarrollo (Sección 06: Modelfile, API y Frontend).